

Documentação de Software

Documentação do Software

Hotel Descanso Garantido – Programa de Gestão Hoteleira

Autores:

Arthur Lima Mendes, Eduardo Lopes Araujo Pêgo, Ítalo Eduardo Carneiro da Silva, Vinicius Matos
Oliveira Rocha

Belo Horizonte
Dezembro, 2025

Documentação do Software

Documentação do Software	2
Introdução	3
Escopo do software	3
Nome do sistema e de seus componentes principais	3
Missão ou objetivo do software	3
Descrição do domínio do Cliente (Regras de Negócio)	4
Funções do produto / Backlog do produto com Histórias de usuário	5
Usuários e sistemas externos.....	6
Descrição	6
Documentação do código.....	7
Documentação da Estrutura de dados geral do software	7
Função void menu().....	12
Função void cadastrarCliente().....	12
Função void cadastrarFuncionario().....	12
Função void cadastrarEstadia().....	12
Testes do software.....	19
Casos de testes do software: função cadastrarQuarto().....	19
Casos de testes do software: função cadastrarCliente()	20
Casos de testes do software: função cadastrarFuncionario().....	20
Casos de testes do software: função cadastrarEstadia().....	21

Introdução

Escopo do software

Nome do sistema e de seus componentes principais

Hotel Descanso Garantido – Programa de Gestão Hoteleira
Componentes:

1. Módulo de Clientes
2. Módulo de Funcionários
3. Módulo de Quartos
4. Módulo de Estádias
5. Módulo de filtrar Clientes
6. Módulo de filtrar Funcionários
7. Módulo de filtrar Estádias
8. Módulo de dar baixa em Estádias
9. Função para exibir MENU principal
10. Funções para salvar os dados em Arquivos Binários
11. Função para gerar códigos únicos para funcionários, clientes e estádias
6. Menu e Controle Principal

Missão ou Objetivo de software

O programa de gestão hoteleira tem como objetivos promover a automatização, organização dos dados de maneira eficiente, agilizar o atendimento, evitar conflitos de reserva e contribuir para a fidelização de clientes. Dessa forma, o setor administrativo do hotel significativamente a ocorrência de erros humanos e atua de maneira mais eficiente e confiável.

Descrição do domínio do Cliente (Regras de Negócio)

Número	Regra de Negócio	Descrição
1	Cadastrar clientes	O sistema deve permitir o cadastro de novos clientes, com os seguintes atributos: código único do cliente, nome, telefone e endereço.
2	Cadastrar funcionários	O sistema deve permitir o cadastro de novos funcionários, com os seguintes atributos: código único do funcionário, nome, telefone e cargo.
3	Cadastrar quarto	O sistema deve permitir o cadastro de novos quartos, com os seguintes atributos: número do quarto, capacidade, valor da diária e descrição do quarto.
4	Cadastrar estadia	O sistema deve permitir cadastrar novas estádias, com os seguintes atributos: código único da estadia, código único do cliente, número de hóspedes, data de checkin e checkout e número do quarto.
5	Validar código do cliente para cadastro de estadia	O sistema deve validar se o código do cliente é válido para prosseguir com cadastro de estadia
6	Verificar quartos disponíveis	O sistema deve verificar os quartos disponíveis de acordo com a data de checkin e checkout informada e exibir na tela todos os quartos disponíveis para prosseguir com a estadia.
7	Exibir dados da estadia ao finalizar cadastro de estádias	O sistema deve exibir previamente os dados da estadia cadastrada, incluindo: código da estadia, código do cliente, data de checkin e checkout, quantidade de diárias e valor.
8	Filtrar clientes	O sistema deve permitir filtrar clientes por: código único do cliente, nome ou endereço e também exibir todos os clientes cadastrados caso necessário e exibir os respectivos dados dos mesmos.
9	Filtrar funcionários	O sistema deve permitir filtrar funcionários por: código único do funcionário, nome ou cargo e também exibir todos os funcionários cadastrados caso necessário e exibir os respectivos dados dos mesmos
10	Filtrar estádias	O sistema deve permitir filtrar estádias de acordo com o código do cliente e também exibir todas as estádias cadastradas no sistema.
11	Dar baixa em estadia	O sistema deve permitir dar baixa em estádias de acordo com o código único da estadia gerado e exibir os dados da estadia e o valor final a ser pago pelo cliente.
12	Calcular pontos de fidelidade dos clientes	O sistema deve calcular os pontos de fidelidade de um cliente toda vez que sua estadia for finalizada, atribuindo 10 pontos de fidelidade para cada dia hospedado.
13	Salvar todos os dados em arquivos binários	O sistema deve salvar todas as informações em arquivos binários, afim de garantir a persistência dos dados.
14	Menu de ações	O sistema deve possuir um menu intuitivo para realizar todas as ações descritas no projeto do software

<i>Número</i>	<i>Regra de Negócio</i>	<i>Descrição</i>
15	As estádias devem ser cadastradas apenas para quartos desocupados	O sistema deve impedir a exibição de quartos que estão ocupados no momento de cadastro de estádias no período específico informado.
16	Edição de atributos do quarto	O sistema deve permitir a alteração de atributos do quarto, como: número do quarto, capacidade, valor da diária e descrição.
17	Listagem de quartos	O sistema deve permitir a listagem de todos os quartos cadastrados no sistema.
18	Dar baixa em estadia	O sistema deve permitir dar baixa em estádias de acordo com o código único da estadia gerado.
19	Verificar código unico de estadia	O sistema deve verificar se o código único de estadia informado no momento de baixa de estadia e valido e caso contrário informar uma mensagem de erro.

Funcionalidades do produto

<i>Número</i>	<i>Funcionalidade do sistema</i>
1	Cadastrar Cliente
2	Cadastrar Funcionário
3	Cadastrar Quarto
4	Editar quarto existente
5	Excluir quarto existente
6	Cadastrar Estádias
7	Filtrar Clientes pelo código, nome ou endereço
8	Filtrar Funcionários pelo código, nome ou cargo
9	Filtrar estádias (todas ou por cliente)
10	Dar baixa em estádias
11	Listar todos os quartos disponíveis
12	Calcular a quantidade de pontos de fidelidade de um cliente

Usuários e sistemas externos Descrição

Número	Usuários	Definição
1	Gerente	Utiliza o sistema para cadastrar funcionários
2	Gerente	Utiliza o sistema para filtrar funcionários
3	Gerente	Utiliza o sistema para cadastrar quartos
4	Gerente	Utiliza o sistema para listar todos os quartos
5	Gerente	Utiliza o sistema para excluir quartos
6	Gerente	Utiliza o sistema para editar quartos
7	Recepcionista	Utiliza o sistema para cadastrar clientes
8	Recepcionista	Utiliza o sistema para cadastrar estádias
9	Recepcionista	Utiliza o sistema para filtrar clientes
10	Recepcionista	Utiliza o sistema para listar as estadias dos clientes
11	Recepcionista	Utiliza o sistema para dar baixa em uma estadia
12	Recepcionista	Utiliza o sistema para mostrar os quartos disponíveis
13	Sistema Interno	Utiliza gerarCodigos() automaticamente ao cadastrar clientes, funcionários e estadias para garantir identificadores únicos.
14	Sistema Interno	Utiliza validaCodigoCliente(), clienteExiste(), funcionarioExiste() e quartoExiste() para validar integridade dos dados antes de operações.
15	Sistema Interno	Utiliza calcularDiarias() automaticamente ao cadastrar estadias para determinar a quantidade de diárias entre check-in e checkout.
16	Sistema Interno	Utiliza verificarDisponibilidadePorPeriodo() e obterQuartosDisponiveisPorPeriodo() para garantir que quartos não sejam reservados em períodos conflitantes.
17	Sistema Interno	Utiliza salvarEstadias(), salvarTodosFuncionarios(), salvarTodosClientes() e salvarQuartoNoArquivo() para salvar dados em arquivos binários.
18	Sistema Interno	Utiliza carregarEstadias(), carregarFuncionarios(), carregarClientes() e lerQuartosDoArquivo() para restaurar dados salvos ao iniciar o programa.
19	Sistema Interno	Utiliza alterarStatusQuarto() automaticamente ao cadastrar ou dar baixa em estadias para atualizar o status dos quartos (ocupado/desocupado).
20	Sistema Interno	Utiliza calcularPontosFidelidade() e adiciona pontos automaticamente ao dar baixa em estadias (10 pontos por diária).

Documentação do código

Documentação da Estrutura de dados geral do software

Explicação sobre bibliotecas, vetores e classes, usados para armazenar e manipular os dados do sistema

Bibliotecas Utilizadas:

#include <iostream>

Fornece funcionalidades para entrada e saída de dados no console, incluindo cin (entrada) e cout (saída).

#include <string>

Fornece a classe string para manipulação de conjunto de caracteres, permitindo operações como concatenação, busca e comparação.

#include <windows.h>

Biblioteca específica do Windows que permite usar funções do sistema operacional, como Sleep() para pausar a execução do programa e SetConsoleOutputCP() para configurar a codificação de caracteres.

#include <fstream>

Permite a manipulação de arquivos (ifstream para leitura e ofstream para escrita), permitindo salvar e carregar dados em arquivos binários.

#include <vector>

Permite armazenar coleções dinâmicas de elementos, usado para armazenar clientes, funcionários, quartos e estadias.

#include <algorithm>

Fornece algoritmos genéricos como transform() para manipulação de sequências, usado para converter strings para minúsculas em buscas case-insensitive.

#include <limits>

Fornece constantes e funções relacionadas aos limites de tipos numéricos, usado para limpar o buffer de entrada.

#include <cstdlib>

Fornece funções utilitárias como system() para executar comandos do sistema operacional (ex: system("cls") para limpar a tela).

#include <ctime>

Fornece funções relacionadas a tempo, como time() e srand() para inicializar o gerador de números aleatórios.

#include <cctype>

Fornece funções para classificação de caracteres, como tolower() para converter caracteres para minúsculas.

#include <cstddef>

Fornece tipos e constantes relacionadas a tamanhos e ponteiros, como size_t.

#include <iomanip>

Fornece manipuladores de formato para entrada/saída, como setprecision() e fixed para formatação de números decimais.

#include <regex>

Fornece suporte para expressões regulares, usado para validar formatos de telefone e endereço.

Classes Declaradas:

Classe Pessoa (Classe Base) - Private

Estrutura de Dados:

int código - Identificador único;

string nome - Nome completo;

string telefone - Número de telefone;

Propósito: Serve como classe base abstrata, definindo atributos comuns a Cliente e Funcionario.

Manipulação: Acessada através de herança pelas classes filhas, que utilizam os métodos get e set para acessar e modificar os atributos.

Classe Cliente - Private

Estrutura de Dados:

int codigo - Código do cliente (herdado de Pessoa)

string nome - Nome do cliente (herdado de Pessoa)

string telefone - Telefone do cliente (herdado de Pessoa)

string endereco - Endereço completo do cliente

int pontosFidelidade - Pontos acumulados no programa de fidelidade

Propósito: Representa um cliente do hotel, armazenando informações pessoais e seus pontos de fidelidade.

Manipulação:

Criação: Instanciada em cadastrarClientes() com dados fornecidos pelo usuário.

Armazenamento: Objetos são armazenados no vetor clientes.

Atualização: Pontos atualizados em darBaixaEstadia() usando adicionarPontosFidelidade().

Persistência: Implementa salvarBinario() e lerBinario() para escrita e leitura em arquivo binário.

Características: Encapsulamento de dados com métodos públicos para acesso controlado.

Classe Funcionario (Herda de Pessoa) - Private

Estrutura de Dados:

Herda: codigo, nome, telefone de Pessoa

float salario - Valor do salário

string cargo - Cargo/função do funcionário

Propósito: Representa um funcionário do hotel, armazenando informações profissionais.

Manipulação:

Criação: Instanciada em cadastrarFuncionarios() com dados fornecidos pelo usuário.

Armazenamento: Objetos são armazenados no vetor funcionarios.

Exibição: Utiliza exibirDados() para apresentar informações formatadas.

Persistência: Implementa salvarBinario() e lerBinario() para persistências do arquivos em binário.

Características: Encapsulamento de dados com métodos para exibição formatada.

Classe Quarto - Private

Estrutura de Dados:

int numQuarto - Número identificador do quarto

int quantidadeHospedes - Capacidade máxima de hóspedes

float valorDiaria - Valor da diária

string status - Status atual ("ocupado" ou "desocupado")

string descricao - Descrição detalhada do quarto

Propósito: Representa um quarto do hotel com todas suas características e estado atual.

Manipulação:

Criação: Instanciada em `cadastrarQuarto()` e `inicializarQuartosFixos()`.

Armazenamento: Não mantido em vetor global, lido do arquivo `quartos.bin` quando necessário.

Busca: Localizado através de `buscarQuartoPorNumero()` e `quartoExiste()`.

Atualização: Status modificado através de `alterarStatusQuarto()` quando estadias são criadas ou finalizadas.

Persistência: Salvo individualmente com `salvarQuartoNoArquivo()` e lido em lote com `lerQuartosDoArquivo()`.

Características: Representa uma entidade persistente que não é mantida em memória globalmente, sendo carregada sob demanda.

Classe Estadia - Private

Estrutura de Dados:

int codigoEstadia - Código único da estadia

int codigoCliente - Código do cliente que fez a reserva

int quantidadeDiarias - Número de diárias

int quantidadeHospedes - Quantidade de hóspedes

int numQuarto - Número do quarto reservado

string dataEntrada - Data de check-in (DD/MM/AAAA)

string dataSaida - Data de checkout (DD/MM/AAAA)

Propósito: Representa uma reserva/estadia no hotel, vinculando cliente, quarto e período.

Manipulação:

Criação: Instanciada em `cadastrarEstadia()` com código gerado automaticamente.

Armazenamento: Objetos são armazenados no vetor `estadias`.

Busca: Localizada por código em `darBaixaEstadia()` ou por cliente em `filtrarEstadiasPorCliente()`.

Remoção: Removida do vetor em `darBaixaEstadia()` quando finalizada.

Persistência: Salva e carregada através de `salvarEstadias()` e `carregarEstadias()` usando escrita/leitura binária.

Características: Representa um registro temporário que liga o cliente ao quarto

Vetores Declarados:

vector<Cliente> clientes

Localização: Variável global

Propósito: Armazena todos os clientes cadastrados no sistema.

Manipulação:

Inserção: Utiliza `push_back()` para adicionar novos clientes quando são cadastrados através de `cadaststrarClientes()`.

Busca: Percorre o vetor usando loops `for` para localizar clientes por código, nome ou endereço nas funções `consultarClientePorCodigo()`, `filtrarClientes()` e `validaCodigoCliente()`.

Atualização: Modifica elementos diretamente quando pontos de fidelidade são adicionados em `darBaixaEstadia()`.

Persistência: Os dados são salvos em `clientes.bin` através de `salvarTodosClientes()` e carregados através de `carregarClientes()`.

Características: Tamanho dinâmico, permite adicionar e remover elementos conforme necessário.

vector<Funcionario> funcionarios

Localização: Variável global

Propósito: Armazena todos os funcionários cadastrados no sistema.

Manipulação:

Inserção: Utiliza `push_back()` para adicionar novos funcionários quando são cadastrados através de `cadaststrarFuncionarios()`.

Busca: Percorre o vetor usando loops `for` para localizar funcionários por código, nome ou cargo nas funções `consultarFuncionarioPorCodigo()`, `filtrarFuncionarios()` e `funcionarioExiste()`.

Persistência: Os dados são salvos em `funcionarios.bin` através de `salvarTodosFuncionarios()` e carregados através de `carregarFuncionarios()`.

Características: Tamanho dinâmico, mantém os dados em memória durante toda a execução do programa.

vector<Estadia> estadias

Localização: Variável global

Propósito: Armazena todas as estadias (reservas) cadastradas no sistema.

Manipulação:

Inserção: Utiliza `push_back()` para adicionar novas estadias quando são cadastradas através de `cadaststrarEstadia()`.

Busca: Percorre o vetor para localizar estadias por código do cliente em `filtrarEstadiasPorCliente()` e `darBaixaEstadia()`.

Remoção: Utiliza `erase()` para remover estadias quando são finalizadas em `darBaixaEstadia()`.

Persistência: Os dados são salvos em `estadias.bin` através de `salvarEstadias()` e carregados através de `carregarEstadias()`.

Características: Tamanho dinâmico, permite gerenciar reservas ativas e históricas.

vector<Quarto> (retornado por funções)

Localização: Não há vetor global de quartos. Os quartos são lidos do arquivo quando necessário.

Propósito: Armazena temporariamente quartos para processamento e exibição.

Manipulação:

Criação: Criado localmente em funções como lerQuartosDoArquivo(), obterQuartosDisponiveis(), obterQuartosDisponiveisPorPeriodo() e listarQuartosDisponiveis().

Filtragem: Utiliza push_back() para adicionar quartos que atendem a critérios específicos (capacidade, disponibilidade, período).

Busca: Percorre o vetor para encontrar quartos específicos ou verificar disponibilidade.

Características: Vetor temporário criados conforme necessário, não mantidos em memória globalmente.

int diasPorMes[]

Localização: Declarado localmente nas funções calcularDiarias() e converterDataParaDias().

Estrutura: int diasPorMes[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

Propósito: Armazena a quantidade de dias em cada mês do ano para cálculos de datas.

Manipulação: Acessado por índice (0-11) para obter dias de cada mês durante conversão de datas para dias desde a data base.

Características: Array estático de tamanho fixo (12 elementos), valores constantes.

Quarto quartos[20]

Localização: Declarado localmente na função `inicializarQuartosFixos()`.

Estrutura: Array de 20 objetos Quarto.

Propósito: Armazena temporariamente os 20 quartos padrão durante a inicialização do sistema.

Manipulação: Preenchido em loops e cada quarto é salvo individualmente no arquivo através de salvarQuartoNoArquivo().

Características: Array estático usado apenas durante inicialização, não mantido após a função.

string descricoes[]

Localização: Declarado localmente na função inicializarQuartosFixos().

Estrutura: Arrays de strings com descrições pré-definidas para os quartos.

Propósito: Armazena descrições padrão para os quartos fixos durante inicialização.

Manipulação: Acessado por índice para atribuir descrições aos quartos durante criação.

Características: Arrays temporários usados apenas durante inicialização.

Vetores Temporários para Resultados de Busca

Exemplos:

vector<Funcionario> resultados em filtrarFuncionarios() - armazena funcionários encontrados em buscas por nome ou cargo.

vector<Cliente*> resultados em filtrarClientes() - armazena ponteiros para clientes encontrados em buscas por nome ou endereço.

vector<Estadia> estadiasFiltradas em filtrarEstadiasPorCliente() - armazena estadias de um cliente específico.

Propósito: Armazenar resultados temporários de operações de busca e filtragem para exibição ao usuário.

Funções do sistema:

Função: `void menu();`

Estrutura de Dados:

Parâmetros: Nenhum.

Retorno: Nenhum.

Propósito: Exibe o menu principal e gerencia a navegação do sistema.

Comportamento: Apresenta 9 opções e executa funções correspondentes. Repete até que o usuário selecione a opção de sair. Exibe erro para opções inválidas.

Função: `void cadastrarCliente();`

Estrutura de Dados:

Parâmetros: Nenhum.

Retorno: Nenhum.

Propósito: Cadastra novos clientes no sistema.

Comportamento: Solicita os dados do cliente, gera um código único, valida endereço e telefone, verifica duplicidade e insere no vetor clientes. Salva no arquivo clientes.bin e inicializa pontos com 0.

Função: `void cadastrarFuncionario();`

Estrutura de Dados:

Parâmetros: Nenhum.

Retorno: Nenhum.

Propósito: Cadastra novos funcionários no sistema.

Comportamento: Solicita dados, gera código, valida salário e verifica duplicidade. Insere no vetor funcionarios e salva no arquivo funcionarios.bin.

Função: `void cadastrarEstadia ();`

Estrutura de Dados:

Parâmetros: Nenhum.

Retorno: Nenhum.

Propósito: Cadastra novas estadias no sistema.

Comportamento: Solicita código do cliente, valida existência, registra datas, calcula diárias, obtém quartos disponíveis, permite escolha do quarto, cria a estadia, altera status para ocupado e salva em estadias.bin.

Função: `void cadastrarQuarto();`

Estrutura de Dados:

Parâmetros: Nenhum.

Retorno: Nenhum.

Propósito: Gerencia o cadastro e edição de quartos.

Comportamento: Abre submenu para cadastrar, listar, editar ou excluir quartos. Valida entradas e salva no arquivo quartos.bin.

Função: `void filtrarClientes();`

Estrutura de Dados:

Parâmetros: Nenhum (os critérios de busca são coletados do usuário).

Retorno: Nenhum.

Propósito: Pesquisar clientes por diferentes critérios (codigo, nome, endereço).

Comportamento: Pesquisa por código, nome ou endereço, ou lista todos. Exibe dados com `exibirDadosCliente`.

Função: `void filtrarFuncionarios();`

Estrutura de Dados:

Parâmetros: Nenhum (os critérios de busca são coletados do usuário).

Retorno: Nenhum.

Propósito: Pesquisar funcionários por diferentes critérios (codigo, nome, cargo).

Comportamento: Permite busca por código, nome, cargo ou listar todos. Utiliza `consultarFuncionarioPorCodigo` e exibe dados usando `exibirDados`.

Função: `bool validaDadosCliente (int clientCode);`

Estrutura de Dados:

Parâmetros: `int clienteCode` – Código do cliente a ser validado

Retorno: `Bool` – Indica se o usuário existe

Propósito: Validar se o código do cliente informado é válido.

Comportamento: Percorre vetor clientes e compara códigos.

Função: `void salvarEstadias ();`

Estrutura de Dados:

Parâmetros: Nenhum

Retorno: Nenhum

Propósito: Persistir todas as estadias em arquivo binário.

Comportamento: Escreve tamanho do vetor e cada estadia no arquivo `estadias.bin`.

Função: `void carregarEstadias ();`

Estrutura de Dados:

Parâmetros: Nenhum

Retorno: Nenhum

Propósito: Carregar as estadias presentes no arquivo `estadia.bin`.

Comportamento: Lê todas as entradas de `estadias.bin` e reconstrói objetos no vetor `global`.

Função: `int calcularDiarias(string dataEntrada, string dataSaida);`

Estrutura de Dados:

Parâmetros: `string dataEntrada`, `string dataSaida`.

Retorno: `int` - Número de diárias calculado, ou 0 se as datas forem inválidas ou a data de saída for anterior à de entrada.

Propósito: Calcular diárias entre duas datas.

Comportamento: Converte ambas as datas para dias e retorna a diferença. Retorna 0 se datas inválidas.

Função: `bool quartoExiste(int numeroQuarto);`

Estrutura de Dados:

Parâmetros: `int numeroQuarto`.

Retorno: `bool` – indica se o quarto existe.

Propósito: Verificar a existência de quarto.

Comportamento: Lê todos os quartos e procura por número correspondente.

Função: `void salvarQuartoNoArquivo(Quarto quarto);`

Estrutura de Dados:

Parâmetros: `Quarto quarto`.

Retorno: Nenhum

Propósito: Salvar os dados do quartos no arquivo.

Comportamento: Escreve dados do quarto no arquivo quartos.bin.

Função: `vector<Quarto> lerQuartosDoArquivo();`

Estrutura de Dados:

Parâmetros: Nenhum

Retorno: `vector<Quarto>`

Propósito: Carregar todos os quartos cadastrados.

Comportamento: Lê sequencialmente todos os quartos do arquivo quartos.bin.

Função: `void limparBuffer();`

Estrutura de Dados:

Parâmetros: Nenhum

Retorno: Nenhum

Propósito: Limpar buffer de entrada.

Comportamento: Limpa erros do cin e descarta caracteres restantes.

Função: `void listarQuartosDisponiveis();`

Estrutura de Dados:

Parâmetros: Nenhum

Retorno: Nenhum

Propósito: Exibir quartos desocupados com capacidade adequada.

Comportamento: Lê a quantidade de hóspedes, valida e filtra quartos com status desocupado.

Função: `void listarTodosQuartos ();`

Estrutura de Dados:

Parâmetros: Nenhum

Retorno: Nenhum

Propósito: Exibir todos os quartos cadastrados.

Comportamento: Mostra todos os quartos e estatísticas como taxa de ocupação.

Função: `void editarQuarto();`

Estrutura de Dados:

Parâmetros: Nenhum

Retorno: Nenhum

Propósito: Editar dados de um quarto Existente.

Comportamento: Permite alterar capacidade, diária e descrição. Reescreve arquivo completo.

Função: `void excluirQuarto();`

Estrutura de Dados:

Parâmetros: Nenhum

Retorno: Nenhum

Propósito: Remover um quarto do sistema.

Comportamento: Valida inexistência de estadias ativas e reescreve o arquivo sem o quarto.

Função: `void excluirQuarto();`

Estrutura de Dados:

Parâmetros: Nenhum

Retorno: Nenhum

Propósito: Remover um quarto do sistema.

Comportamento: Valida inexistência de estadias ativas e reescreve o arquivo sem o quarto.

Função: `vector<Quarto> obterQuartosDisponiveis(int quantidadeHospedes)`

Estrutura de Dados:

Parâmetros: `int` quantidadeHospedes.

Retorno: `vector<Quarto>`.

Propósito: Listar quartos desocupados por capacidade.

Comportamento: Filtra quartos com status desocupado e capacidade suficiente.

Função: `bool alterarStatusQuarto(int numeroQuarto, string novoStatus)`

Estrutura de Dados:

Parâmetros: `int` numeroQuarto, `string` novoStatus.

Retorno: `bool` – `true` se a alteração foi bem-sucedida, `false` se o quarto não foi encontrado ou houve erro ao salvar.

Propósito: Alterar status do quarto.

Comportamento: Atualiza status e reescreve o arquivo quartos.bin.

Função: `bool verificarDisponibilidadePorPeriodo(int numQuarto, string dataCheckin, string dataCheckout)`

Estrutura de Dados:

Parâmetros: `int` numQuarto, `string` dataCheckin, `string` dataCheckout.

Retorno: `bool` – `true` se o quarto está disponível no período, `false` caso contrário.

Propósito: Verificar disponibilidade em um período.

Comportamento: Checa status e verifica conflito de datas.

Função: `vector<Quarto> obterQuartosDisponiveisPorPeriodo(int quantidadeHospedes, string dataCheckin, string dataCheckout)`

Estrutura de Dados:

Parâmetros: `int` quantidadeHospedes, `string` dataCheckin, `string` dataCheckout.

Retorno: `vector<Quarto>` - Vetor contendo quartos disponíveis no período.

Propósito: Filtrar quartos disponíveis por período.

Comportamento: Considera capacidade e usa verificarDisponibilidadePorPeriodo.

Função: `void darBaixaEstadia();`

Estrutura de Dados:

Parâmetros: Nenhum (o código da estadia é solicitado ao usuário).

Retorno: Nenhum.

Propósito: Finalizar estadia.

Comportamento: Libera quarto, adiciona pontos de fidelidade, remove estadia e salva atualizações.

Função: `void listarTodasEstadias();`

Estrutura de Dados:

Parâmetros: Nenhum.

Retorno: Nenhum.

Propósito: Exibir todas as estadias cadastradas.

Comportamento: Lista todas ou filtra por cliente, exibindo dados formatados.

Função: `Quarto buscarQuartoPorNumero(int numeroQuarto)`

Estrutura de Dados:

Parâmetros: `int` numeroQuarto.

Retorno: `Quarto` - Objeto quarto encontrado, ou um quarto vazio (número 0) se não encontrado.

Propósito: Encontrar um quarto específico.

Comportamento: Percorre lista de quartos e retorna o encontrado.

Função: `void inicializarQuartosFixos();`

Estrutura de Dados:

Parâmetros: Nenhum.

Retorno: Nenhum.

Propósito: Criar quartos iniciais caso arquivo esteja vazio.

Comportamento: Cria 20 quartos fixos com valores predefinidos.

Função: `int calcularPontosFidelidade(int codigoCliente)`

Estrutura de Dados:

Parâmetros: `int` codigoCliente.

Retorno: `int` - Quantidade de pontos de fidelidade do cliente, ou 0 se o cliente não for encontrado.

Propósito: Obter pontos de fidelidade acumulados de um cliente.

Comportamento: Busca cliente e retorna seus pontos de fidelidade.

Função: `void salvarTodosFuncionario();`

Estrutura de Dados:

Parâmetros: Nenhum.

Retorno: Nenhum.

Propósito: Salvar funcionários em arquivo binário.

Comportamento: Escreve tamanho e chama salvarBinario para cada funcionário.

Função: `void carregarFuncionarios();`

Estrutura de Dados:

Parâmetros: Nenhum.

Retorno: Nenhum.

Propósito: Ler funcionários do arquivo.

Comportamento: Lê tamanho e usa `lerBinario` para reconstruir objetos.

Função: `Funcionario* consultarFuncionarioPorCodigo(int codigoBusca)`

Estrutura de Dados:

Parâmetros: `int codigoBusca`.

Retorno: `Funcionario*`.

Propósito: Buscar funcionário específico.

Comportamento: Percorre o vetor global `funcionarios`.

Função: `bool funcionarioExiste(int codigoBusca)`

Estrutura de Dados:

Parâmetros: `int codigoBusca`.

Retorno: `bool` – `true` se o funcionário existe, `false` caso contrário.

Propósito: Verificar existência de funcionário.

Comportamento: Usa `consultarFuncionarioPorCodigo` e verifica ponteiro.

Função: `Cliente* consultarClientePorCodigo(int codigoBusca)`

Estrutura de Dados:

Parâmetros: `int codigoBusca`.

Retorno: `Cliente*` - Ponteiro para o cliente encontrado, ou `nullptr` se não encontrado.

Propósito: Buscar cliente específico.

Comportamento: Percorre vetor `clientes`.

Função: `void exibirDadosCliente(const Cliente& cliente);`

Estrutura de Dados:

Parâmetros: `const Cliente& cliente`.

Retorno: Nenhum.

Propósito: Exibir dados do cliente.

Comportamento: Mostra tabela com código, nome, telefone, endereço e pontos.

Função: `void salvarTodosClientes();`

Estrutura de Dados:

Parâmetros: Nenhum.

Retorno: Nenhum.

Propósito: Persistir clientes.

Comportamento: Escreve tamanho do vetor e chama `salvarBinario` dos clientes.

Função: `void carregarClientes();`

Estrutura de Dados:

Parâmetros: Nenhum.

Retorno: Nenhum.

Propósito: Carregar clientes do arquivo.

Comportamento: Lê tamanho e usa `lerBinario` para reconstruir objetos.

Função: `int clienteExiste(int codigoBusca)`

Estrutura de Dados:

Parâmetros: `int codigoBusca`.

Retorno: `bool` – `true` se o cliente existe, `false` caso contrário

Propósito: Verificar existência de cliente.

Comportamento: Percorre vetor global e verifica códigos.

Função: *string* lerEndereco()

Estrutura de Dados:

Parâmetros: Nenhum.

Retorno: *string* - Endereço válido no formato especificado.

Propósito: Ler e validar endereço.

Comportamento: Solicita entrada do usuário e valida por regex.

Função: *string* lerTelefone()

Estrutura de Dados:

Parâmetros: Nenhum.

Retorno: *string* - Telefone válido no formato "31 9xxxx-xxxx".

Propósito: Validar telefone do cliente

Comportamento: Solicita entrada e valida formato "31 9xxxx-xxxx".

Função: *int* converterDataParaDias(*string* data)

Estrutura de Dados:

Parâmetros: *string* data.

Retorno: *int* - Número de dias desde 01/01/2000, ou 0 se a data for inválida.

Propósito: Converter datas em dias desde 01/01/2000.

Comportamento: Extrai dia/mês/ano, valida e calcula conversão.

Função: *bool* haConflitoDatas(*string* checkin1, *string* checkout1, *string* checkin2, *string* checkout2)

Estrutura de Dados:

Parâmetros: *string* checkin1, *string* checkout1, *string* checkin2, *string* checkout2.

Retorno: *bool* – *true* se há conflito *false* caso contrário.

Propósito: Verificar sobreposição de períodos.

Comportamento: Converte datas e verifica conflito entre intervalos.

Função: *void* filtrarEstadiaPorCliente()

Estrutura de Dados:

Parâmetros: Nenhum.

Retorno: Nenhum.

Propósito: Filtrar estadias por cliente.

Comportamento: Permite busca por nome ou código e exibe estadias relacionadas.

Função: *void* limparTela

Estrutura de Dados:

Parâmetros: Nenhum.

Retorno: Nenhum.

Propósito: Limpar tela do console.

Comportamento: Executa system("cls").

Função: *int* main();

Estrutura de Dados:

Parâmetros: Nenhum.

Retorno: *int* – Código de saída do programa (return 0);

Propósito: Ponto de entrada do programa.

Comportamento: Inicializa ambiente, carrega dados, chama menu e salva alterações ao encerrar.

Testes do software***Casos de testes do software: função cadastrarQuarto()***

<i>Número</i>	<i>Variáveis de entrada</i>	<i>Valores válidos</i>	<i>Resultados esperados</i>	<i>Valores inválidos</i>	<i>Resultado esperado</i>
1	Número do quarto	101	Quarto cadastrado com sucesso	-1	Erro: número inválido
2	Quantidade de hóspedes	2	""	0	Erro: quantidade inválida
3	Valor diário	150,00	""	-10	Erro: status inválido
4	Status	Desocupado		""	Erro: status inválido
5	Descrição	"Quarto teste"	Cadastro salvo no arquivo	""	Erro: descrição vazia
6	Número do quarto	101, 102	Verificação do quarto 101 retorna <i>true</i> ; Quarto 102 salvo; Total = 2	101 (duplicado)	Erro: quarto já existente (não salva)
7	Quantidade de hóspedes	2, 3	2, 3	-	-
8	Valor diário	150.00, 200.00	Valores aceitos	-	-
9	Número do quarto	102	Cadastro aceito	-	-
10	Quantidade de hóspedes	4 (máximo permitido)	Hóspedes = 4	-	-
11	Valor diário	250.00	Valor aceito	-	-

Casos de testes do software: função cadastrarCliente()

Número	Varáveis de Entrada	Valores válidos	Resultado Esperado	Valores inválidos	Resultado Esperado
1	Código	12345	Cliente salvo no arquivo; total = 1	-	-
2	Nome	"João Silva"	Nome armazenado corretamente	-	-
3	Endereço	"Rua A (123), Bairro Centro, Cidade Belo Horizonte"	Endereço salvo	-	-
4	Telefone	"31999999999"	Telefone salvo	-	-
5	Pontos fidelidade	0	Pontos = 0	-	-
6	Código	12345	Cliente encontrado (ponteiro ≠ null)	-	-
7	Nome armazenado	"Maria Santos"	Retornado corretamente	-	-
8	Busca parcial por nome	"Maria"	Retorna true (cliente encontrado)	-	-

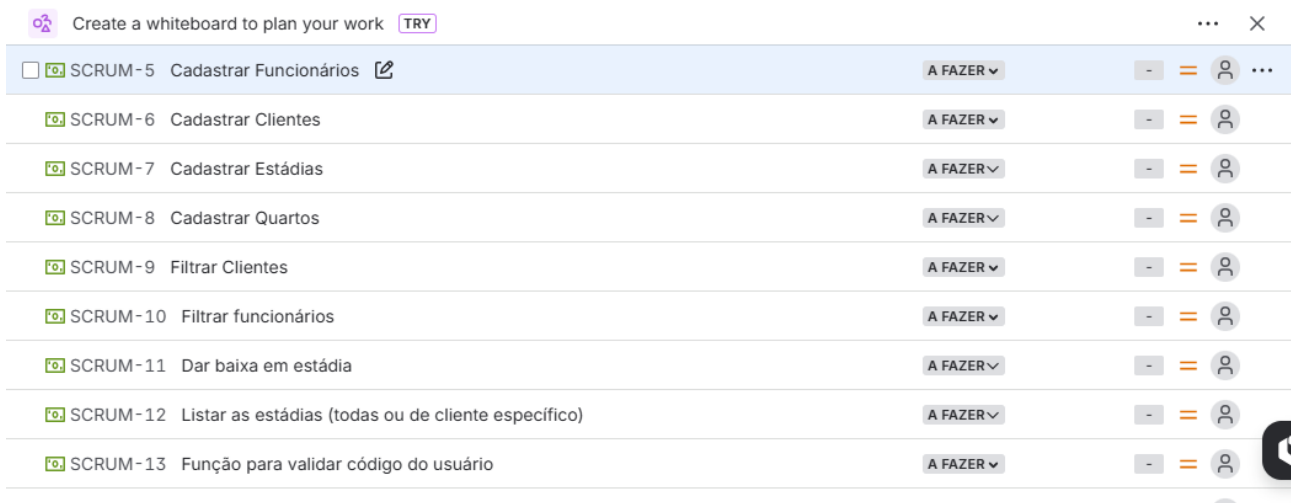
Casos de testes do software: função cadastrarFuncionario()

Número	Varáveis de Entrada	Valores válidos	Resultado Esperado	Valores inválidos	Resultado Esperado
1	Código	10001	Funcionário salvo no arquivo; total	-	-
2	Nome	"João Funcionario"	Nome armazenado corretamente	-	-
3	Telefone	"31999999999"	Telefone salvo	-	-
4	Cargo	"Recepcionista"	Cargo salvo	-	-
5	Salário	2000,00	Salário salvo	-	-
6	Código	10002	Funcionário encontrado (ponteiro ≠ null)	-	-

Casos de testes do software: função cadastrarEstadiao()

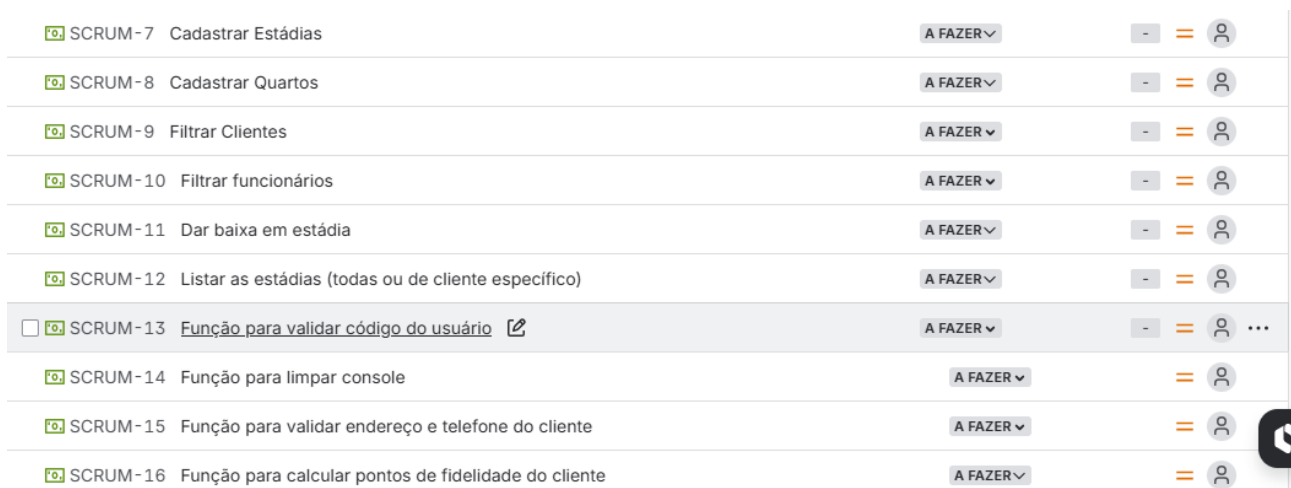
<i>Número</i>	<i>Varáveis de Entrada</i>	<i>Valores válidos</i>	<i>Resultado Esperado</i>	<i>Valores inválidos</i>	<i>Resultado Esperado</i>
1	Cliente	Código 12345, Nome "João Silva"	Cliente associado corretamente à estadia	-	-
2	Quarto	Nº 101, capacidade 2, diária 150,00, status "desocupado"	Quarto disponível para reserva	-	-
3	Estadia	2 hóspedes, entrada 12/01/2025, saída 12/03/2025, 2 diárias	Estadia cadastrada com sucesso	-	-
4	Ação	Marcar quarto como ocupado	Quarto muda para status "ocupado"	-	-
5	-	-	Quantidade de reservas = 1 Cliente = 12345 Quarto = 101 Diárias = 2 Status = "ocupado"	-	-
6	Quarto existente	-	-	-	-
7	Estadia atual	-	-	-	-
8	Período solicitado	12/03/2025 – 12/06/2025 (conflitante)	Disponível = false	-	-

Registros do backlog do produto e divisão de sprints



☐	SCRUM-5	Cadastrar Funcionários	A FAZER	-	=	...
☐	SCRUM-6	Cadastrar Clientes	A FAZER	-	=	
☐	SCRUM-7	Cadastrar Estádias	A FAZER	-	=	
☐	SCRUM-8	Cadastrar Quartos	A FAZER	-	=	
☐	SCRUM-9	Filtrar Clientes	A FAZER	-	=	
☐	SCRUM-10	Filtrar funcionários	A FAZER	-	=	
☐	SCRUM-11	Dar baixa em estadia	A FAZER	-	=	
☐	SCRUM-12	Listar as estádias (todas ou de cliente específico)	A FAZER	-	=	
☐	SCRUM-13	Função para validar código do usuário	A FAZER	-	=	

Figura 1.0 – Primeira parte do backlog do produto



☐	SCRUM-7	Cadastrar Estádias	A FAZER	-	=	
☐	SCRUM-8	Cadastrar Quartos	A FAZER	-	=	
☐	SCRUM-9	Filtrar Clientes	A FAZER	-	=	
☐	SCRUM-10	Filtrar funcionários	A FAZER	-	=	
☐	SCRUM-11	Dar baixa em estadia	A FAZER	-	=	
☐	SCRUM-12	Listar as estádias (todas ou de cliente específico)	A FAZER	-	=	
☐	SCRUM-13	Função para validar código do usuário	A FAZER	-	=	...
☐	SCRUM-14	Função para limpar console	A FAZER	-	=	
☐	SCRUM-15	Função para validar endereço e telefone do cliente	A FAZER	-	=	
☐	SCRUM-16	Função para calcular pontos de fidelidade do cliente	A FAZER	-	=	

Figura 2.0 – Continuação do backlog do produto

Spaces

TP-AEDS-AIVE

...

Summary

Backlog

Board

Code

Timeline

Pages

Forms

+

Search backlog...

Filter

SCRUM Sprint 0

16 Nov – 23 Nov (4 work items)

0

0

0

Complete sprint

...

SCRUM-5

Cadastrar Funcionários

EM ANDAMENTO

-

=

SCRUM-6

Cadastrar Clientes

EM ANDAMENTO

-

=

SCRUM-7

Cadastrar Estádias

EM ANDAMENTO

-

=

SCRUM-8

Cadastrar Quartos

EM ANDAMENTO

-

=

+

Create

Figura 3.0 – Sprint 0

Especificação dos Requisitos do Software

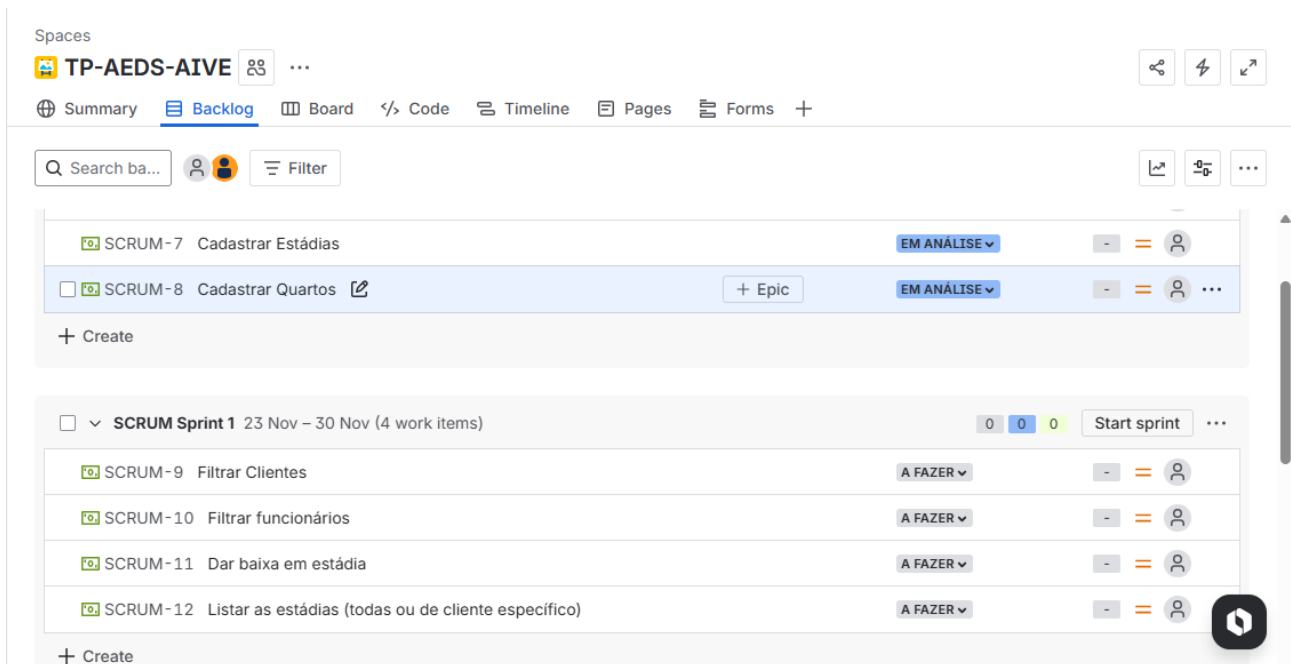


Figura 4.0 – sprint1

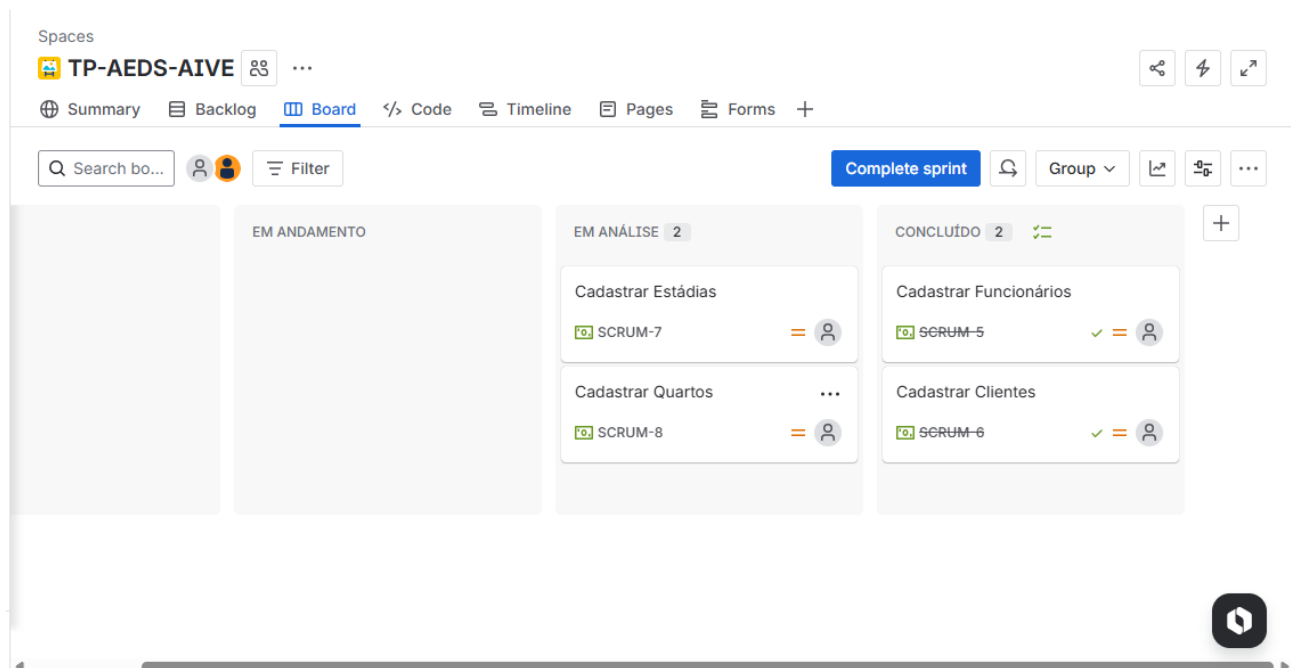


Figura 5 – Analise de sprint

Programa de Gestão de Hotelaria – Hotel Descanso Garantido

Registros das daily meeting realizadas

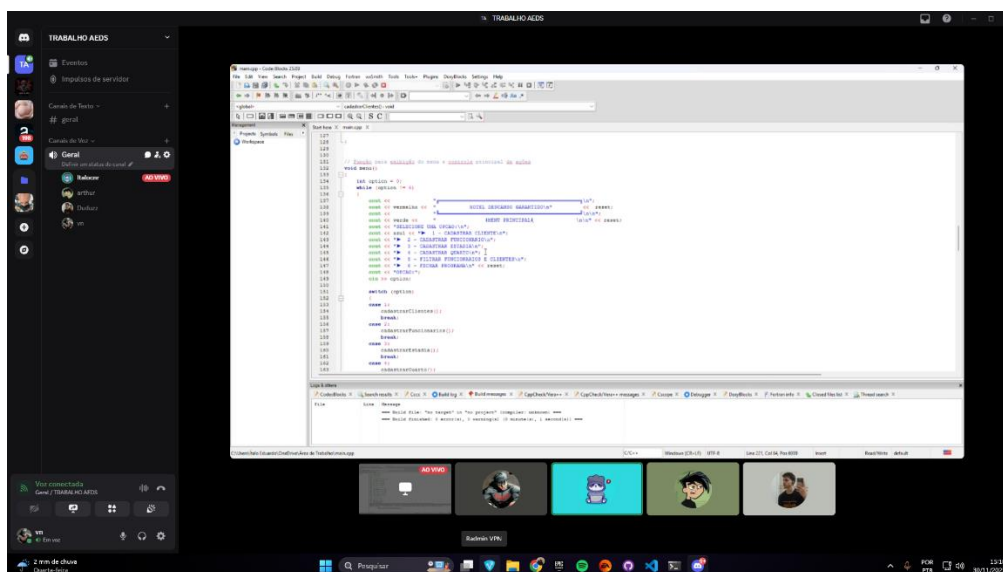


Figura 1.0 – Daily 1

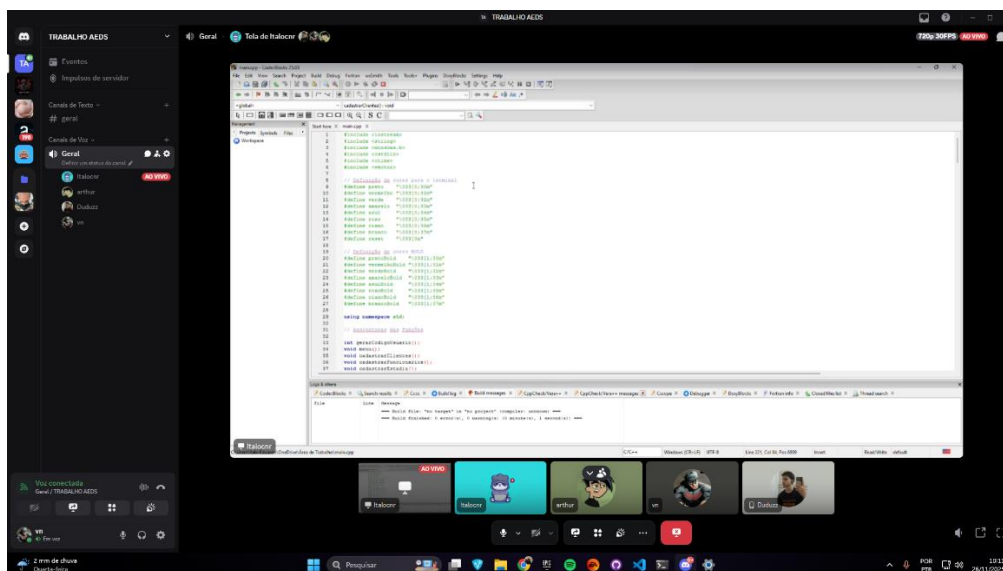


Figura 2.0 – Daily 2

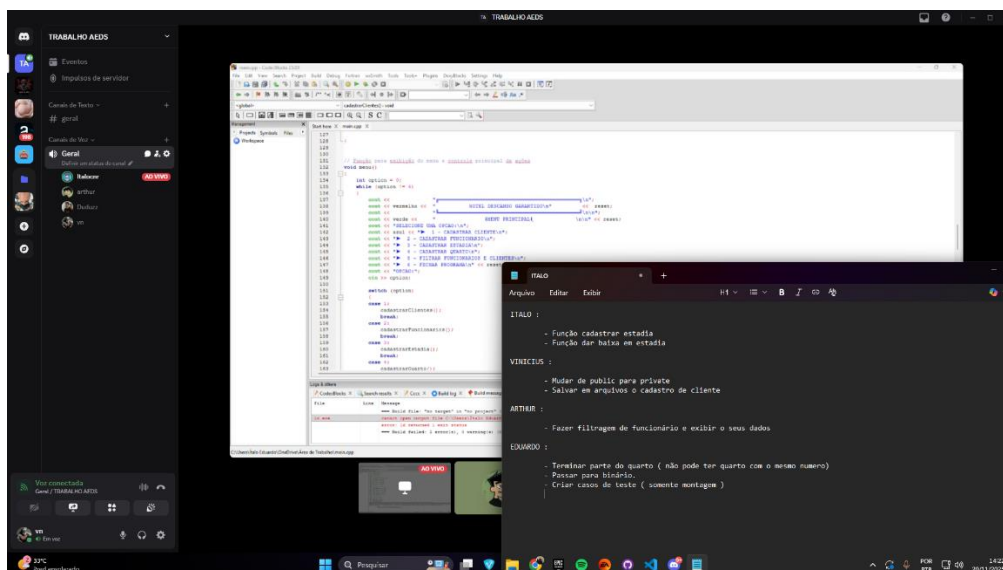


Figura 3.0 – Daily 3